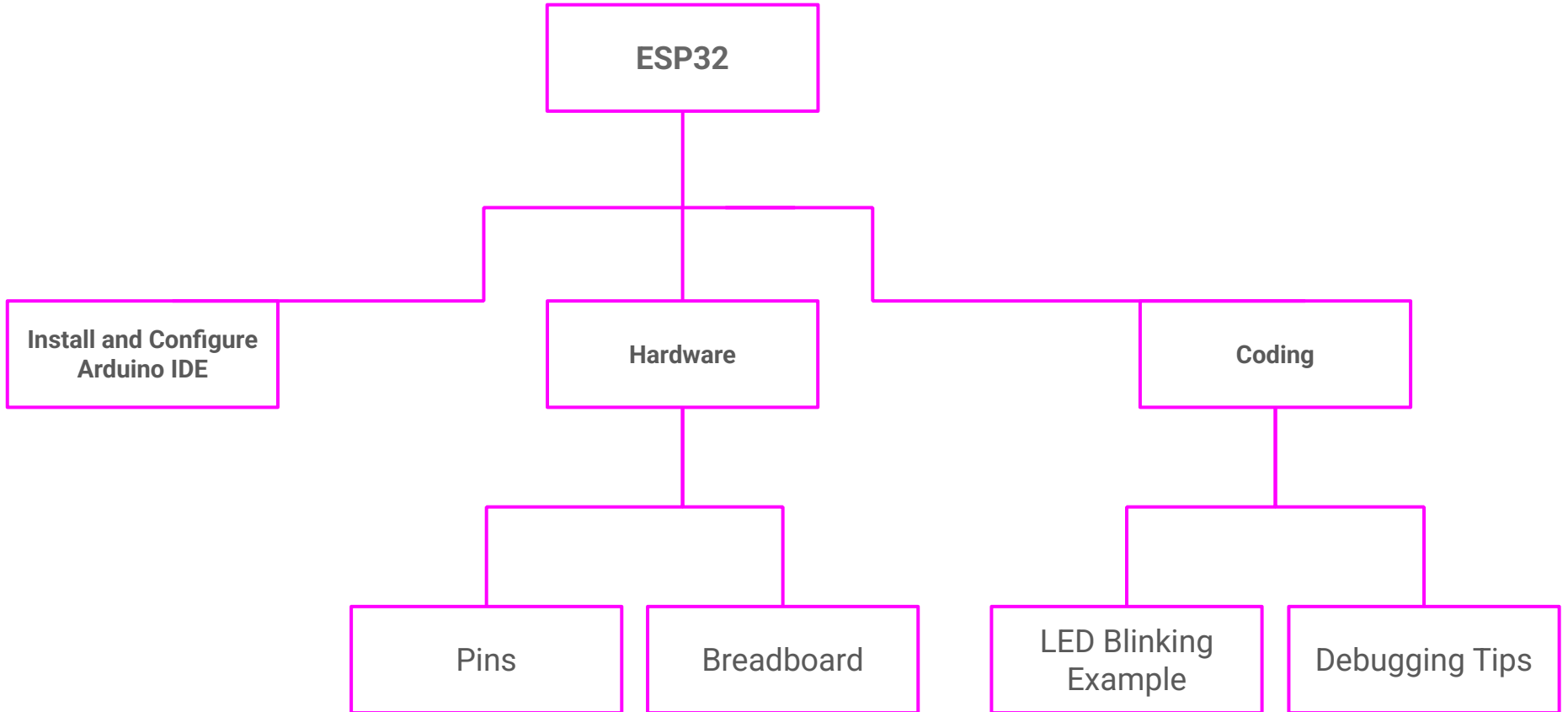


Lab 1 Introduction to ESP32 and Digital I/O

EE/CSE 474
Summer 2026

Learning Objective(s)



Downloading Arduino

Install and Configure
Arduino IDE

1. Navigate to Arduino's software website: <https://www.arduino.cc/en/software>
2. Download it:

Downloads



Arduino IDE 2.3.4

The new major release of the Arduino IDE is faster and even more powerful! In addition to a more modern editor and a more responsive interface it features autocompletion, code navigation, and even a live debugger.

For more details, please refer to the [Arduino IDE 2.0 documentation](#).

Nightly builds with the latest bugfixes are available through the section below.

SOURCE CODE

The Arduino IDE 2.0 is open source and its source code is hosted on [GitHub](#).

DOWNLOAD OPTIONS

Windows Win 10 and newer, 64 bits

Windows MSI installer

Windows ZIP file

Linux AppImage 64 bits (X86-64)

Linux ZIP file 64 bits (X86-64)

macOS Intel, 10.15: "Catalina" or newer, 64 bits

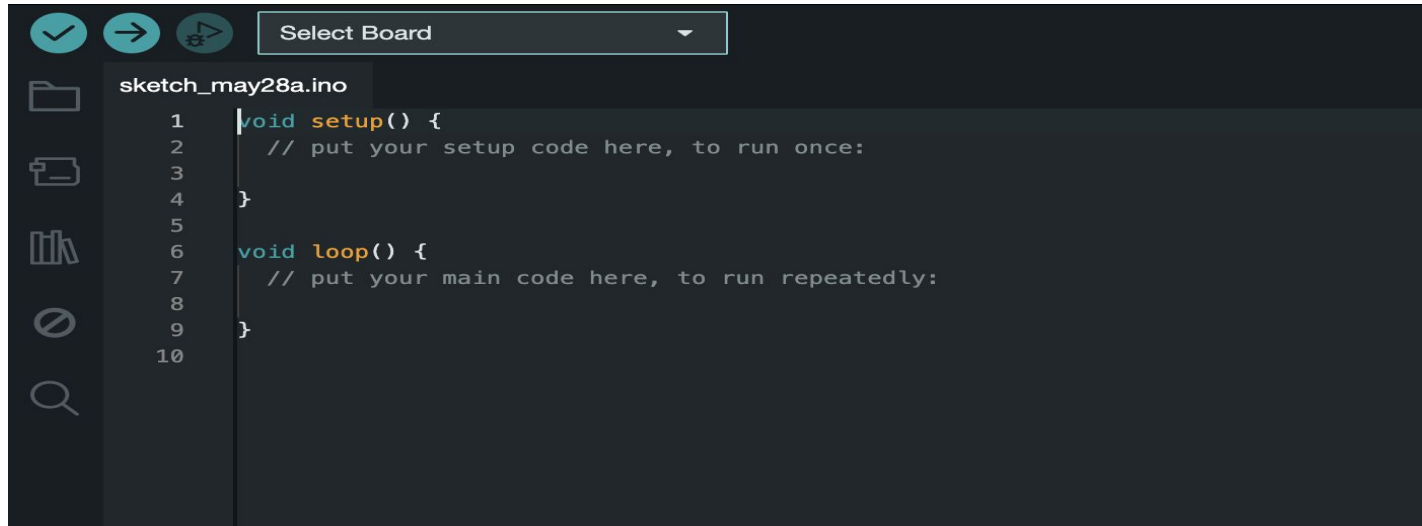
macOS Apple Silicon, 11: "Big Sur" or newer, 64 bits

[Release Notes](#)

Downloading Arduino Cont

Install and Configure
Arduino IDE

3. Once you click your chosen Download Option, you'll be directed to a contribution page and then a newsletter signup. On both pages scroll down and **select "Just Download"**
4. Once your file has downloaded, click to open and start the installation/setup process. You should get something like this:



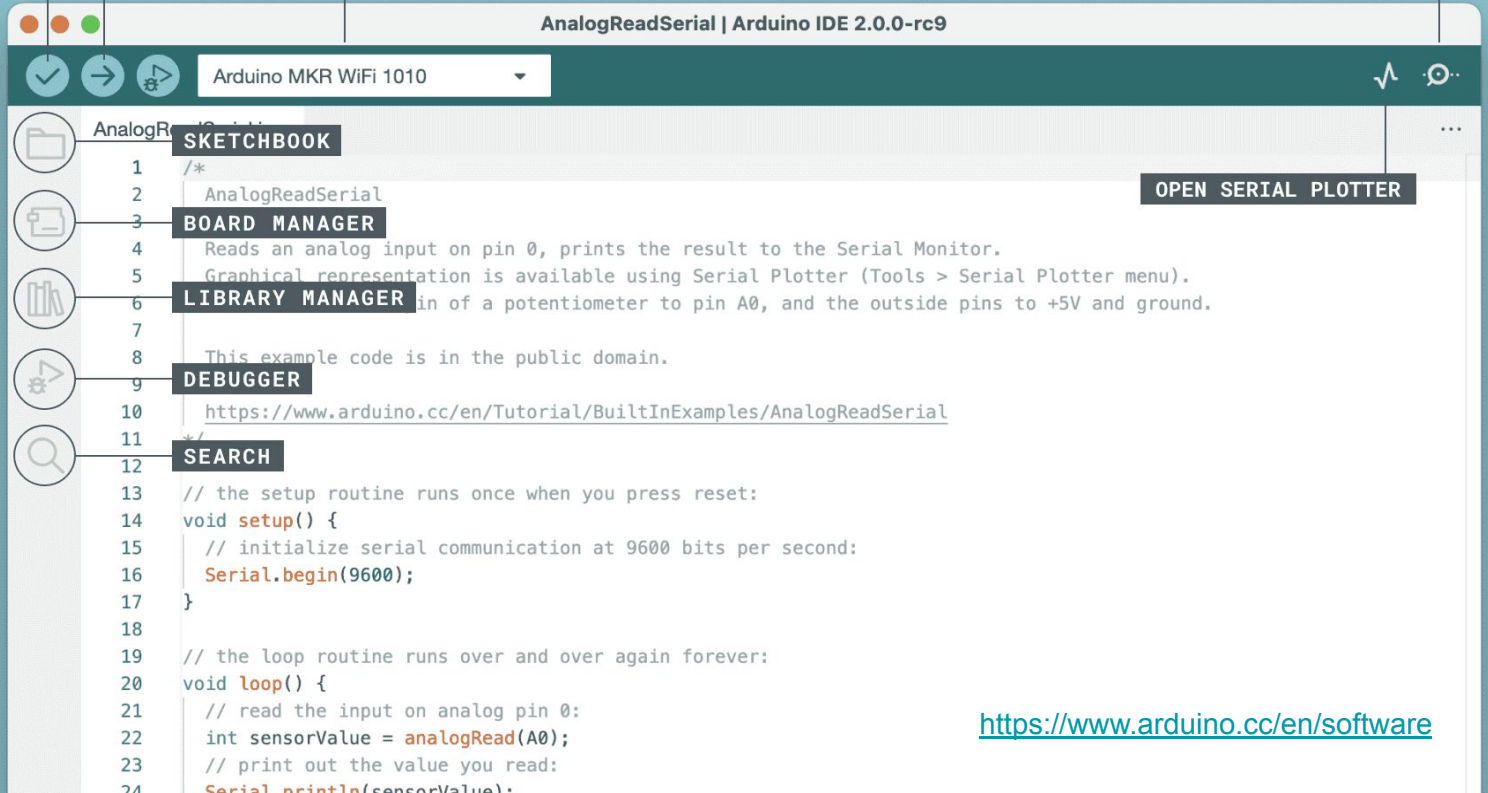
Arduino IDE

Install and Configure
Arduino IDE

VERIFY/UPLOAD

SELECT BOARD & PORT

OPEN SERIAL MONITOR



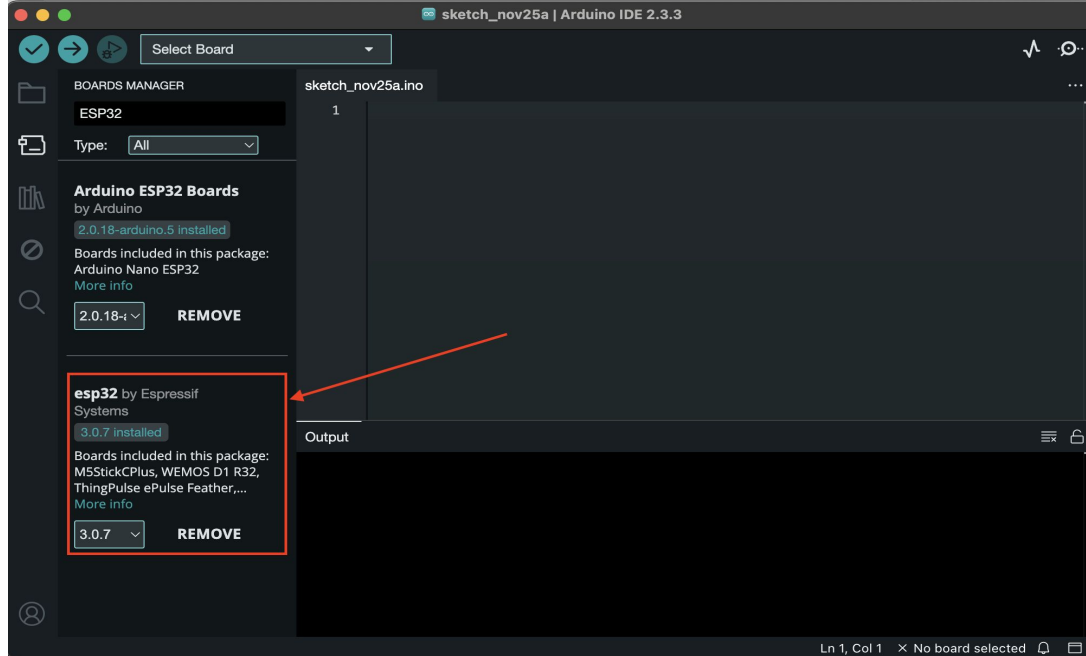
OPEN SERIAL PLOTTER

<https://www.arduino.cc/en/software>

Configuring Arduino

Install and Configure
Arduino IDE

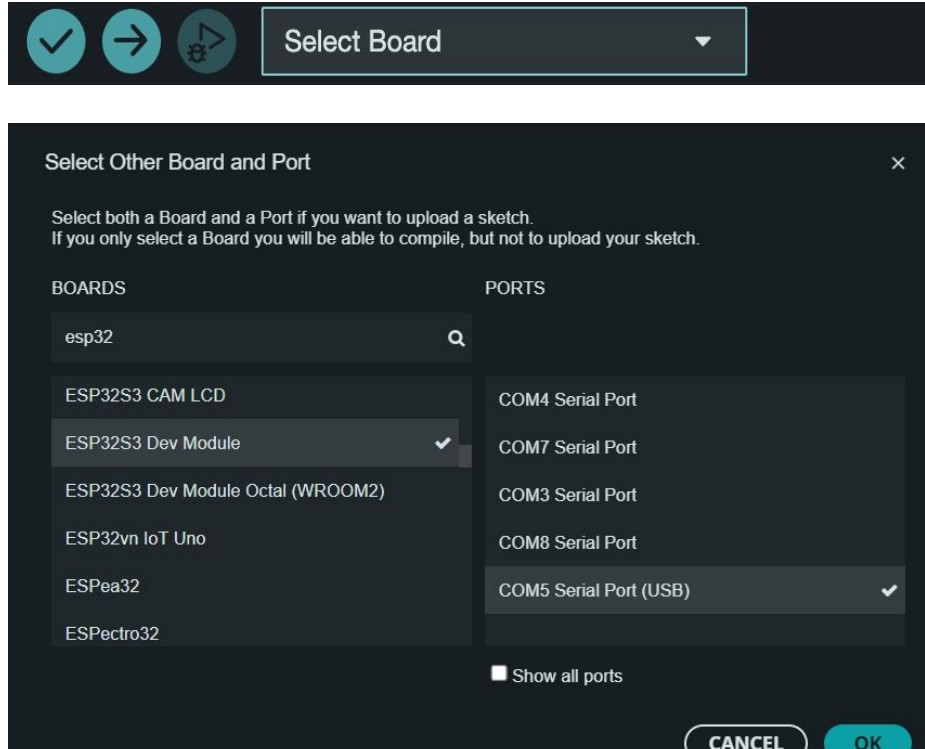
1. With the Arduino IDE open, go to Tools > Board > Boards Manager
2. In the search bar type "ESP32" and you'll notice "ESP32 Boards by espressif" pop up (which covers our board, the ESP32-S3-DEVKITC-1). Click install



Configuring Arduino Cont.

Install and Configure
Arduino IDE

3. Now, click "Select Board" at the top and select the ESP32S3 Dev Module and its COM.

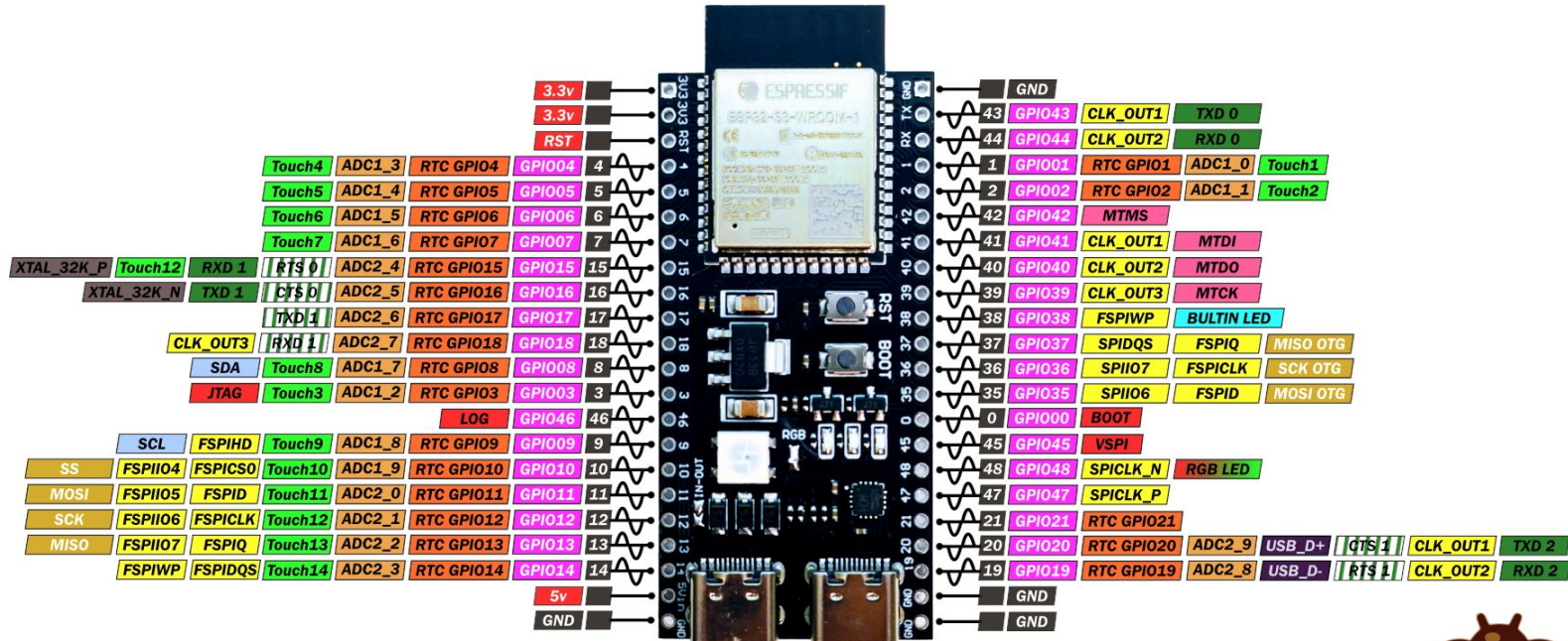


Overview of ESP32-S3

Pins

VCC-GND Studio YD-ESP32-S3 (ESP32-S3-DevKitC-1 clone)

PINOUT



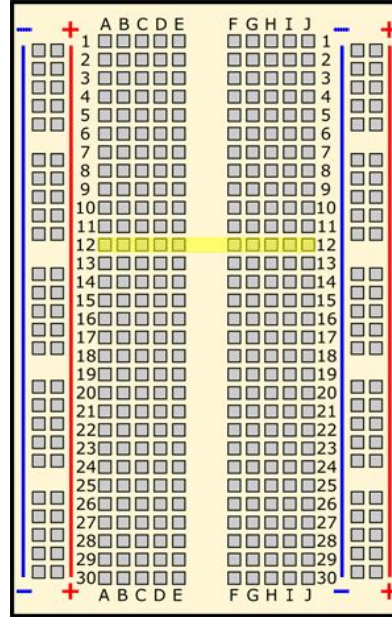
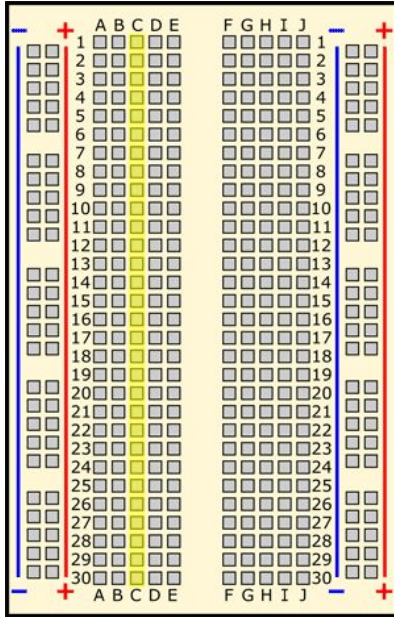
Feature Highlights/Notes

Pins

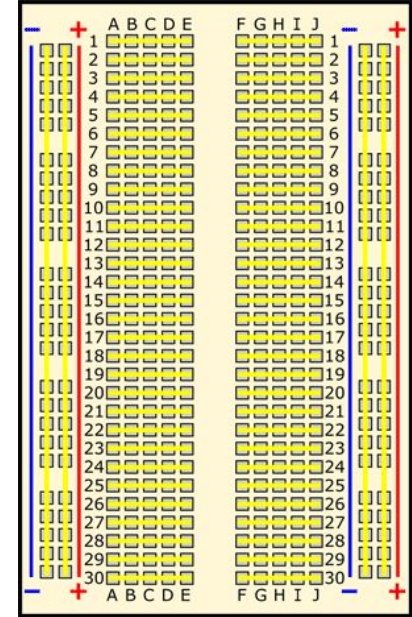
- Pins are labeled at the bottom
- Most things are labeled on the board
- G pins are ground and you should use them in your design
- GPIO pins are not automatically pin low or pin high, that's something you have to code in
- The device has two ways to upload (UART and USB). You can really use either but there are slight code differences in certain scenarios for uploads. This won't affect you in lab 1

Introduction to Breadboarding

Breadboard and Others



Numbering of Rows and Columns



Underlying Connections

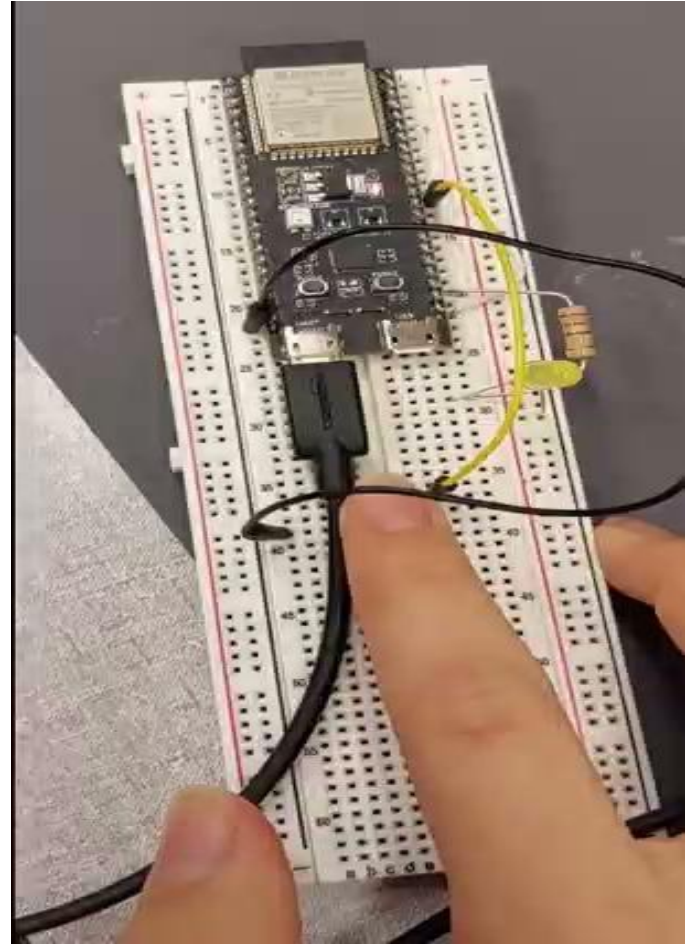
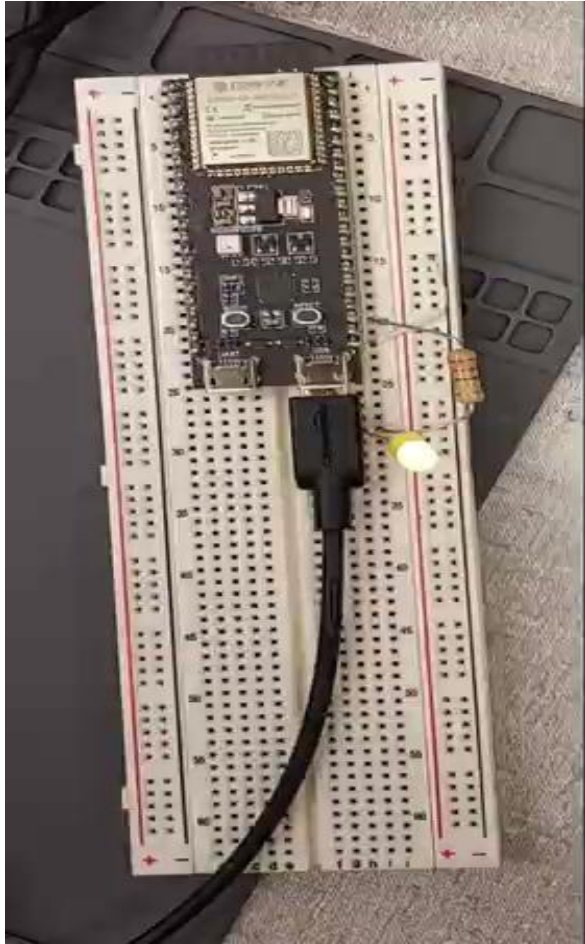
LAB 1 INFO & DEMO

General Lab 1 info

- **When's the deadline?**
 - a. Lab 1 has been released
 - b. June 29th at 11:59pm
 - c. **Allowing Demo on Monday 29th in class**
- **When and how can you pick up your lab kits?**
 - b. You can pick it up from the teaching team
- **Where is the lab located?**
 - b. Room 345 in ECE building. You should have access to it 24/7 with your husky card
- **Only Lab1 is done individually**

Let's do a demo!

LED Blinking Example



What to submit for Lab 1?

Live Demos:

- *Part I:* External (or built-in) LED blinking pattern on the ESP32
- *Part II:* Button-controlled LED operation
- *Part III:* Multiple button-controlled LEDs (Optional)

Code:

NOTE: Please follow the [Code Guidelines](#) when documenting your code

The sketch files for the following sections should be descriptively named (e.g. Lab1Part2.ino), zipped together in a single folder containing your last name(s) and the lab number (e.g. LastNameLab1.zip), and submitted through Canvas:

- *Part I:* External (or built-in) LED blinking pattern on the ESP32
- *Part II:* Button-controlled LED operation
- *Part III:* Multiple button-controlled LEDs (Optional)

Report(s)

- No lab report for Lab 1
- Future Labs will have a report. Use Template provided on Canvas
- Expectations on report
 - Your report should maintain a *professional tone*, adhere to proper formatting guidelines, and include correctly labeled figures and neatly formatted tables. The template on canvas provides the format and some suggestions on potential content to guide your report.
 - *Focus on your discussions and insights.* Avoid restating provided information such as materials and methodology. Instead, emphasize the reasoning behind your approaches, the results obtained, and the lessons learned.
 - *Adhere to the provided template for your report.* You are encouraged to utilize generative AI tools to enhance your writing, provided that the final work remains your own. Clearly cite the usage of any AI tools, such as converting bullet points into paragraphs. Ensure that you can demonstrate authorship of your work if required.

Debugging

1. Check your hardware
 - a. Check if you are connected to your microcontroller
 - b. Check your wiring
 - c. Check if your hardware is working
2. Check your Code
 - a. Check if you are using the correct version of your code
 - b. Check the logic of your code. Make state diagrams if needed to visualize your code
 - c. Add print statements
 - d. Add code in blocks. Use Macros. Initialize all your constants, and variables
3. Use available tools for troubleshooting
 - a. Arduino and ESP-32 have a very strong support community. You won't be the first a issues. Odds are someone else has already found a solution. arduino.cc and [stackoverflow](https://stackoverflow.com) are very popular forums
 - b. AI tools like GPT are very useful for troubleshooting code. Use them.
 - c. Your TAs are valuable resource. Ask them for help if you tried the above steps and you can't identify where the problem is. However, please do not expect the teaching team to give you the answer. We will tell you where you are going wrong, and give you pointers on best idea to fix it. You have to fix it yourself
4. Shut down your computer and Restart.

